

```
"""
```

```
UMA_IR – Intermediate Representation.
```

```
Sits between SemanticEngine (compiler/planner) and UMA kernel (physics).  
SemanticEngine emits a UMA_IR graph. UMAExecutor consumes it.
```

```
Node types:
```

```
'evolve'      – run dynamics forward by dt  
'observe'     – apply (Observation, y) to the filter  
'constrain'   – project state onto constraint set  
'checkpoint'  – record friction, check closure
```

```
"""
```

```
from __future__ import annotations  
from dataclasses import dataclass, field  
from typing import Any, Callable, Dict, List, Optional  
import numpy as np
```

```
@dataclass
```

```
class IRNode:
```

```
    node_id: str  
    kind: str          # 'evolve' | 'observe' | 'constrain' | 'checkpoint'  
    payload: Dict[str, Any] = field(default_factory=dict)
```

```
@dataclass
```

```
class UMA_IR:
```

```
    """Compiled execution graph. Compiler emits this; Executor consumes it."""
```

```
    nodes: List[IRNode] = field(default_factory=list)  
    schedule: List[str] = field(default_factory=list)  
    objective: Optional[Callable[[np.ndarray], float]] = None  
    metadata: Dict[str, Any] = field(default_factory=dict)
```

```
    def append(self, node: IRNode) -> None:
```

```
        self.nodes.append(node)  
        self.schedule.append(node.node_id)
```

```
    def __len__(self) -> int:
```

```
        return len(self.schedule)
```

```
    def summary(self) -> str:
```

```
        counts: Dict[str, int] = {}  
        for n in self.nodes:
```

```
counts[n.kind] = counts.get(n.kind, 0) + 1
parts = [f"{k}×{v}" for k, v in counts.items()]
return f"UMA_IR({', '.join(parts)}, total={len(self)} nodes)"
```